

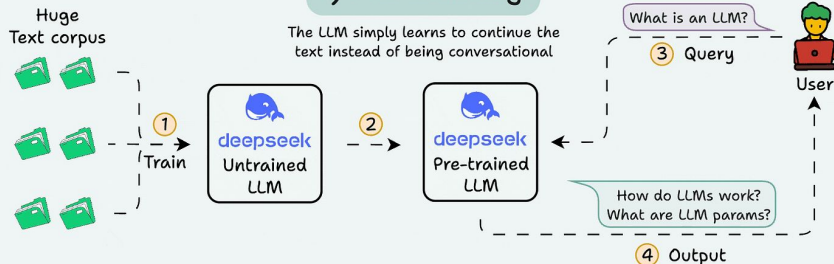
Transformers in Deep Learning

Lecture 7

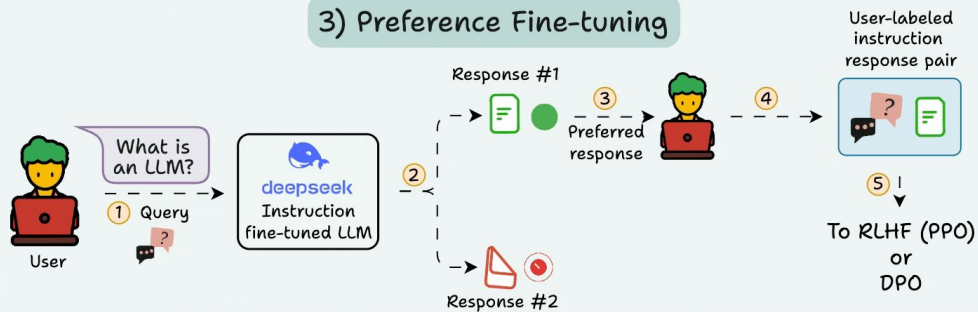
Lecture 6 Recap

1) Pre-training

The LLM simply learns to continue the text instead of being conversational

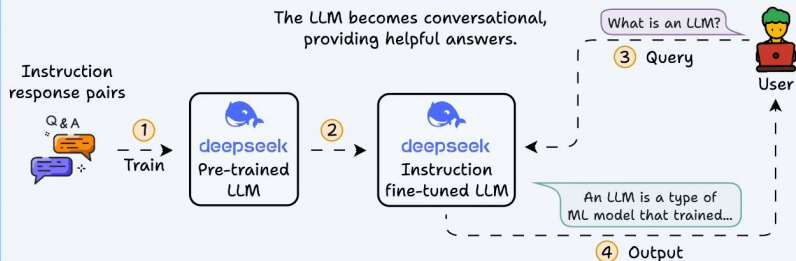


3) Preference Fine-tuning



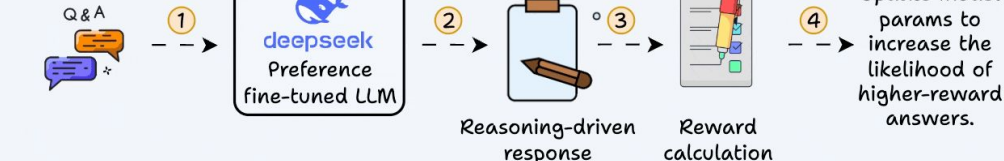
2) Instruction fine-tuning

The LLM becomes conversational, providing helpful answers.



4) Reasoning fine-tuning

Reasoning task with a definitive answer



Lecture 6 Recap

Basics

Tokenization
Architecture
Loss function
Optimizer
Learning rate

Systems

Kernels
Parallelism
Quantization
Activation checkpointing
CPU offloading
Inference

Scaling laws

Scaling sequence
Model complexity
Loss metric
Parametric form

Data

Evaluation
Curation
Transformation
Filtering
Deduplication
Mixing

Alignment

Supervised fine-tuning
Reinforcement learning
Preference data
Synthetic data
Verifiers

Overview

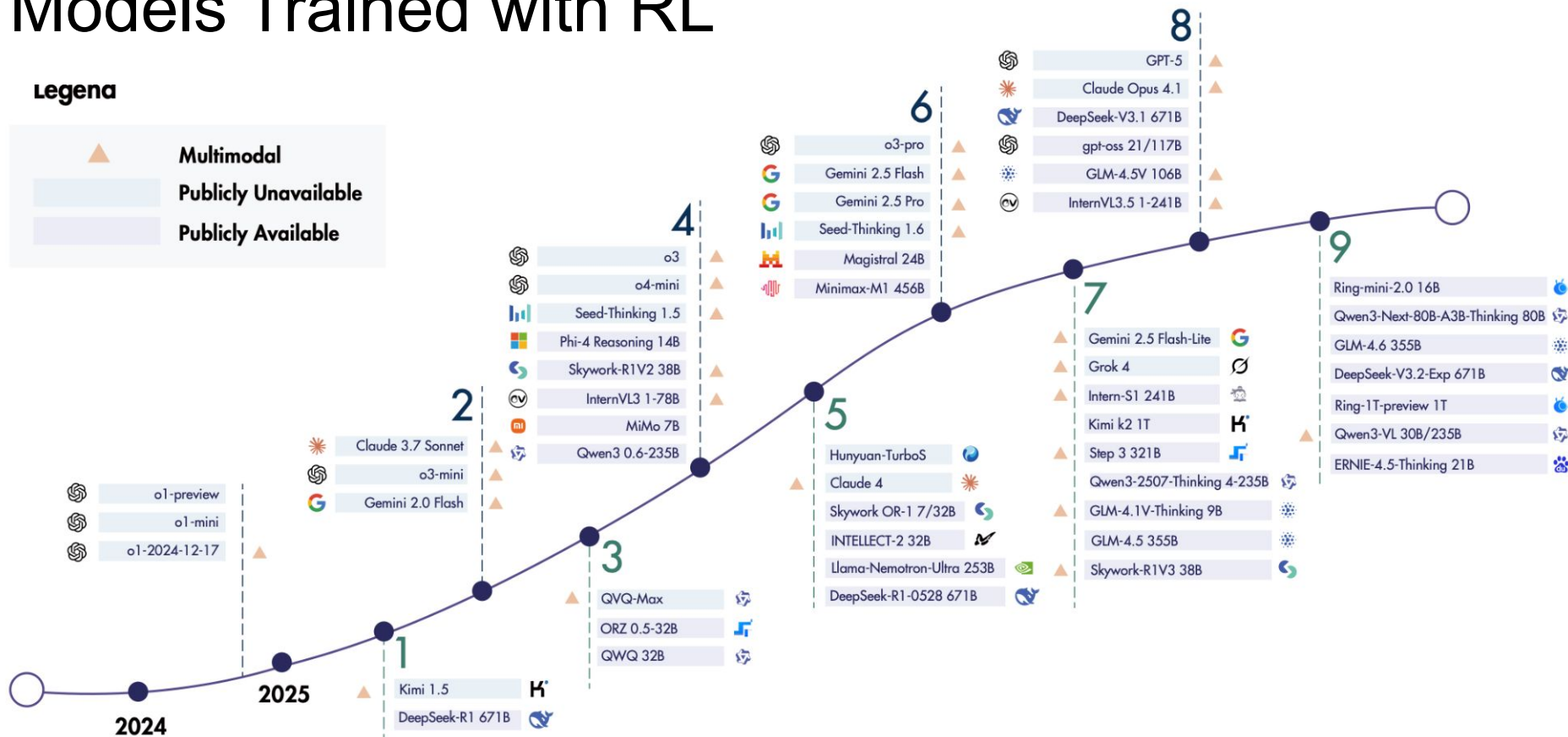
- 1) RL
- 2) PEFT (Lora, Adapters, Prompt\Prefix tuning)
- 3) Pruning
- 4) How computer store model weights?
- 5) Mixed Precision Training
- 6) Quantization
- 7) Distillation
- 8) Distributed GPU Training (**next lecture**)
- 9) Efficient Attention (**next lecture**)
- 10) MoE (**next lecture**)
- 11) Inference Triton and etc. (**next lecture**)

RL

Models Trained with RL

Legenda

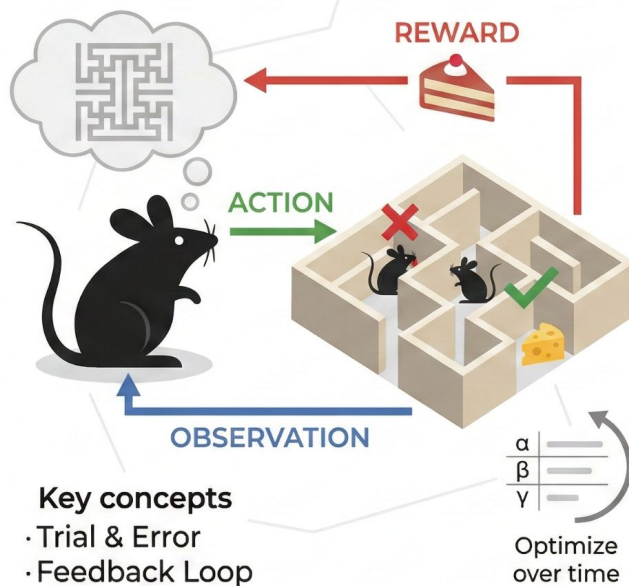
▲	Multimodal
■	Publicly Unavailable
■	Publicly Available



RL (Reinforcement Learning)

REINFORCEMENT LEARNING (RL)

Learning via Experience & Trial



RL (Reinforcement Learning)

REINFORCEMENT LEARNING (RL)

Learning via Experience & Trial

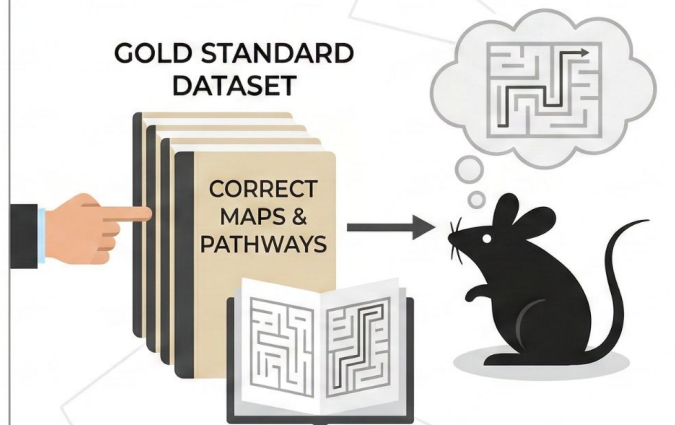


Key concepts

- Trial & Error
- Feedback Loop

SUPERVISED FINE-TUNING (SFT)

Learning via Examples & Imitation



Key concepts

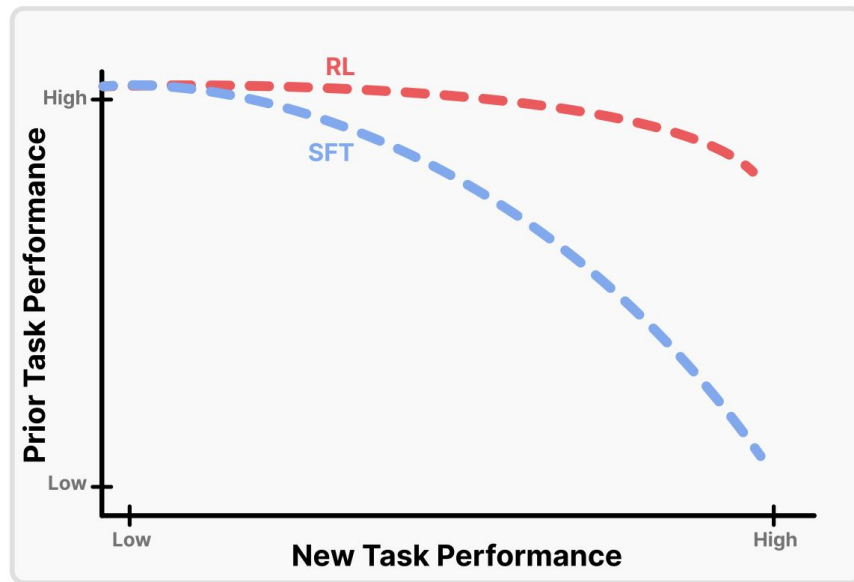
- Dataset Memorization
- Golden Examples
- Imitation-Based

oop

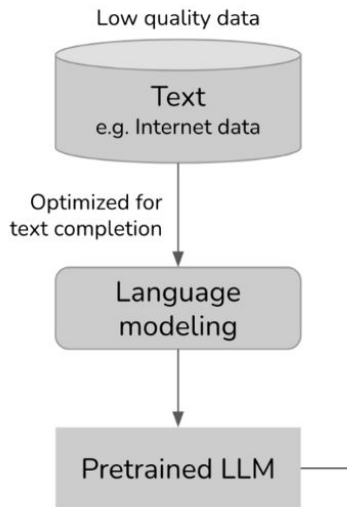
Not used directly
for learning

Why RL?

- 1) Learning from negative feedback
- 2) RL forgets less
- 3) Better Generalization
- 4) Better Exploration
- 5) Reasoning and Self-Correction



RLHF (Preference Tuning)



Scale
May '23

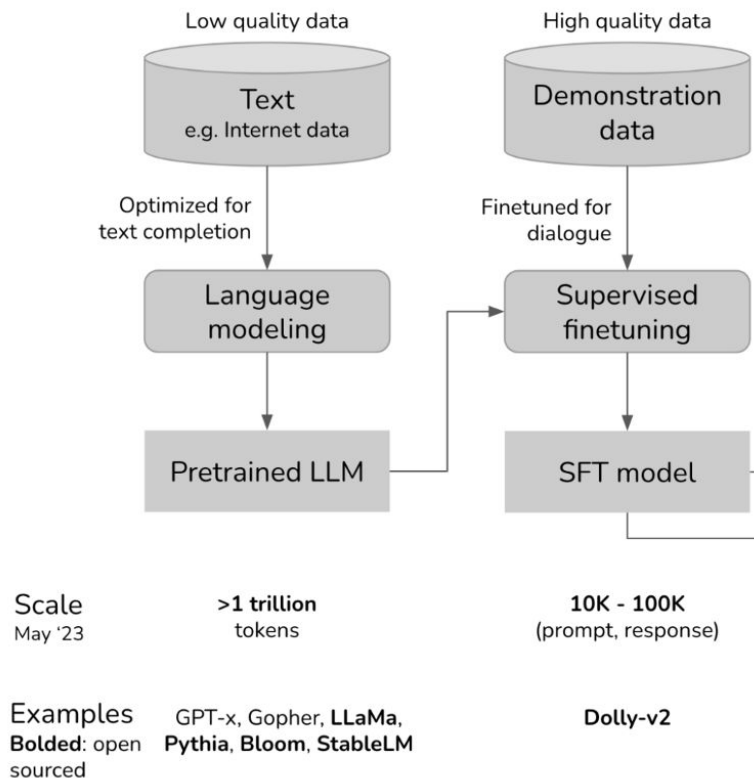
>1 trillion
tokens

Examples

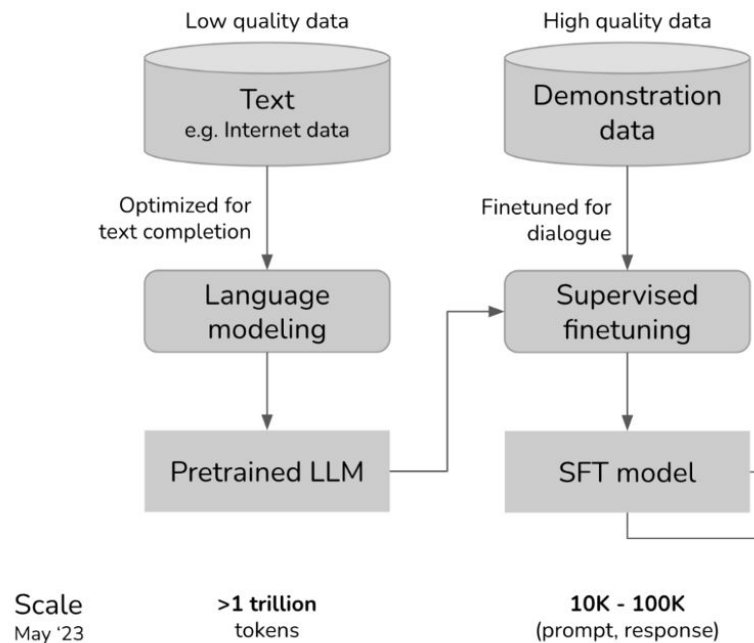
Bolded: open
sourced

GPT-x, Gopher, **LLaMa**,
Pythia, **Bloom**, **StableLM**

RLHF (Preference Tuning)



RLHF (Preference Tuning)

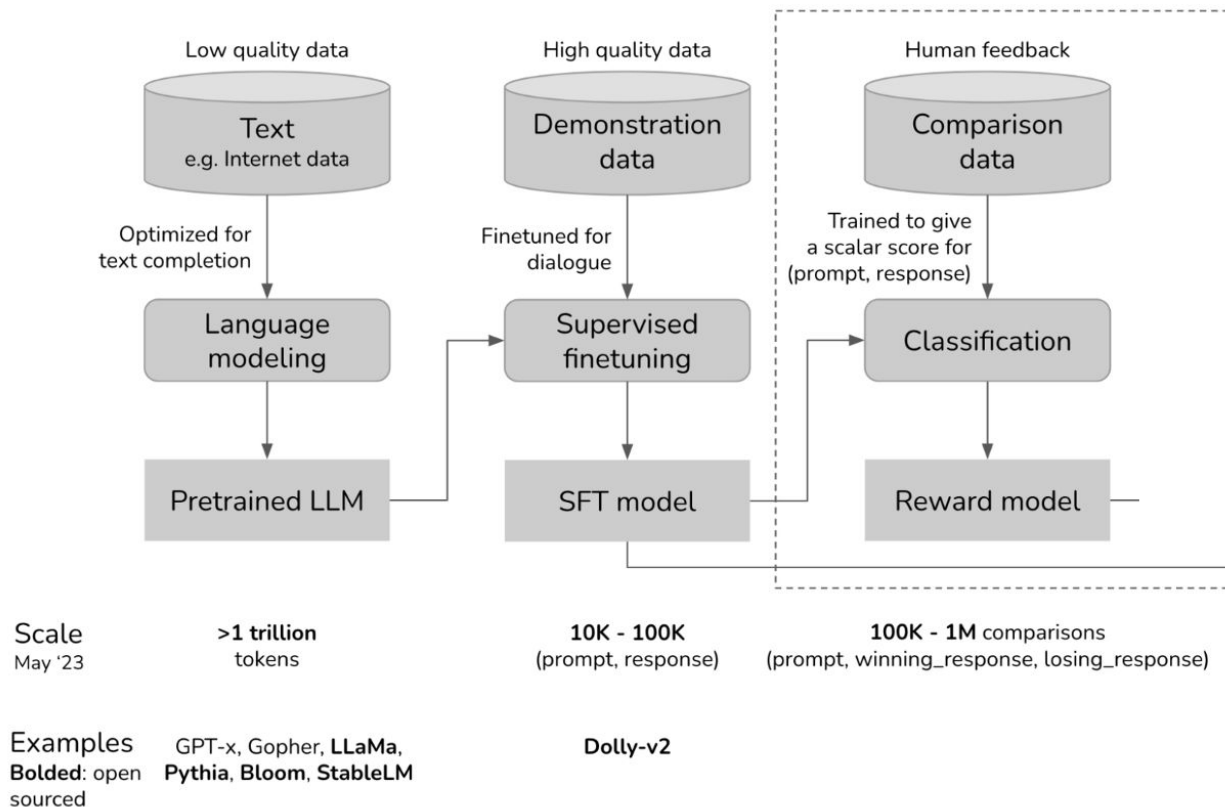


Examples
Bolded: open sourced
GPT-x, Gopher, **LLaMa**,
Pythia, **Bloom**, **StableLM**

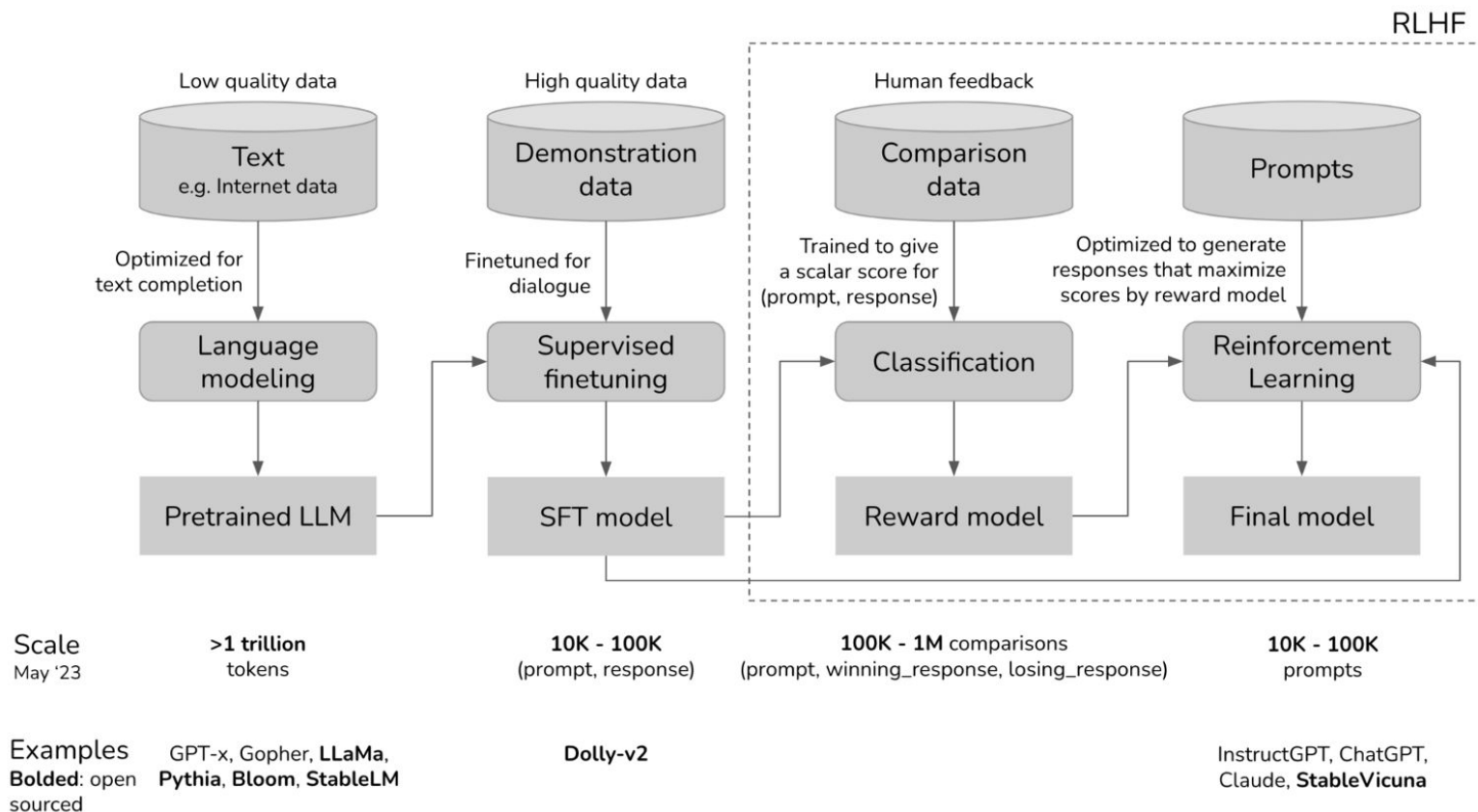
Dolly-v2

	RedPajama	LLaMA*
CommonCrawl	878 billion	852 billion
C4	175 billion	190 billion
Github	59 billion	100 billion
Books	26 billion	25 billion
ArXiv	28 billion	33 billion
Wikipedia	24 billion	25 billion
StackExchange	20 billion	27 billion
Total	1.2 trillion	1.25 trillion

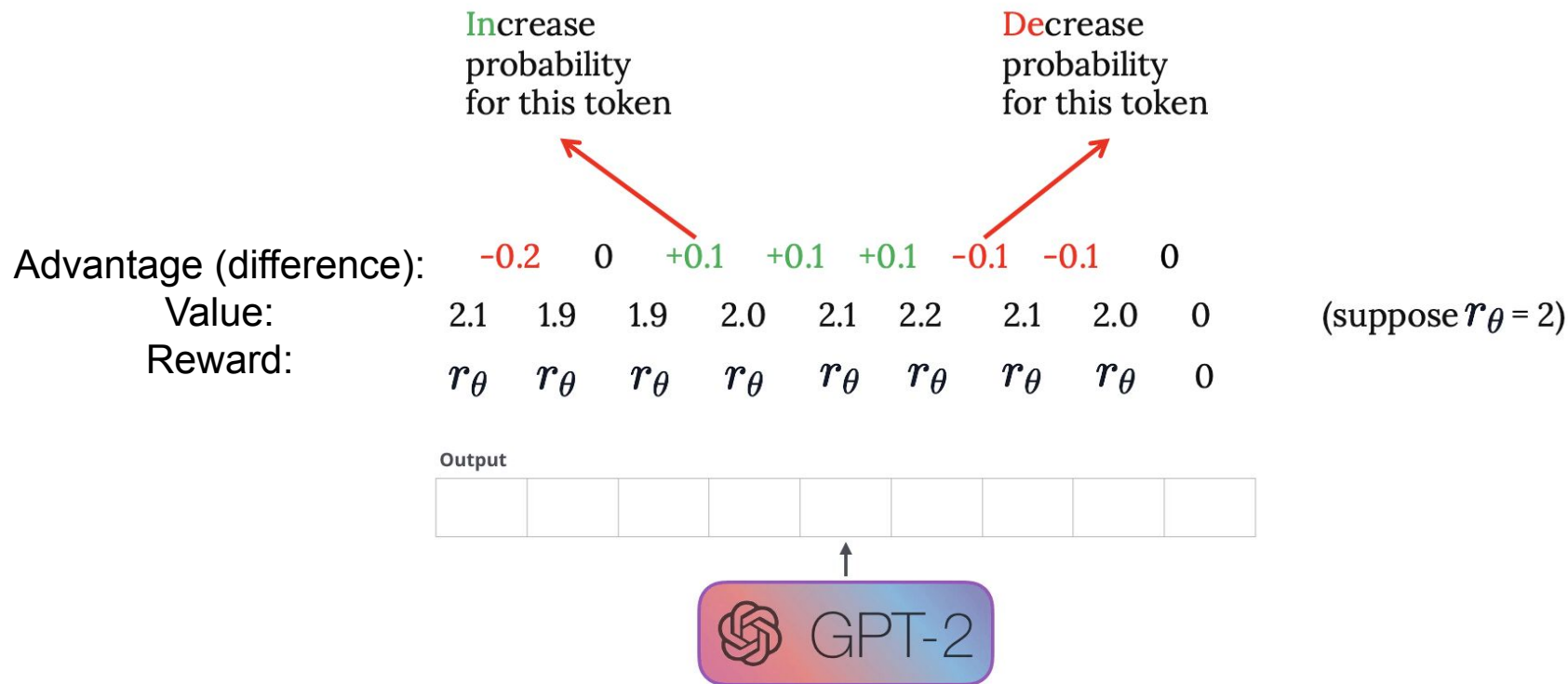
RLHF (Preference Tuning)



RLHF (Preference Tuning)

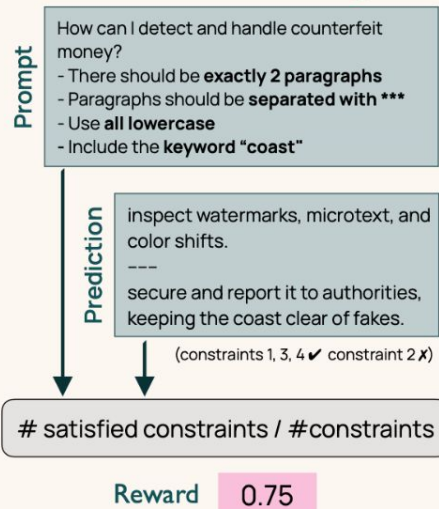


RLHF (Preference Tuning)

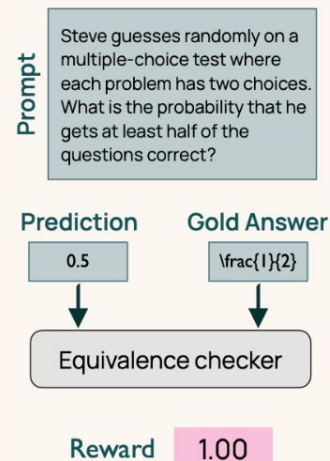


Verifiable Reward

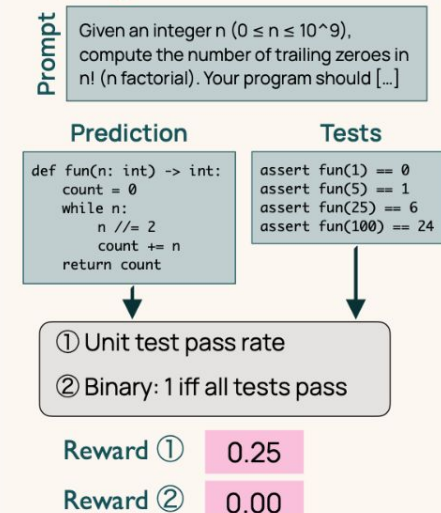
Instruction Following



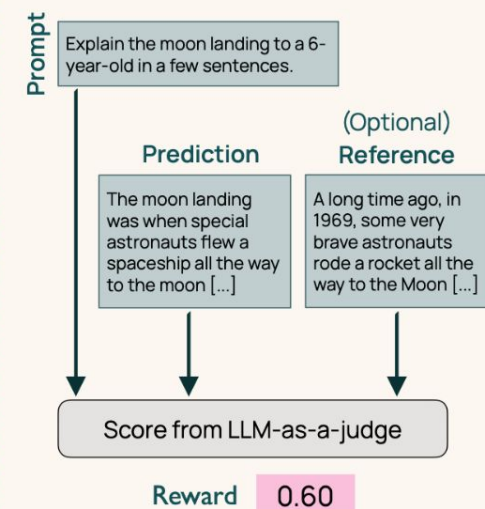
Math



Coding



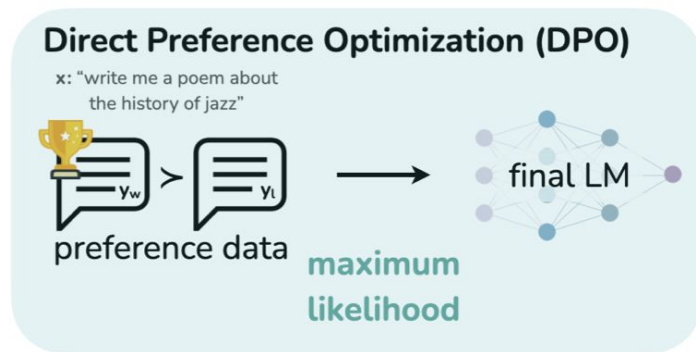
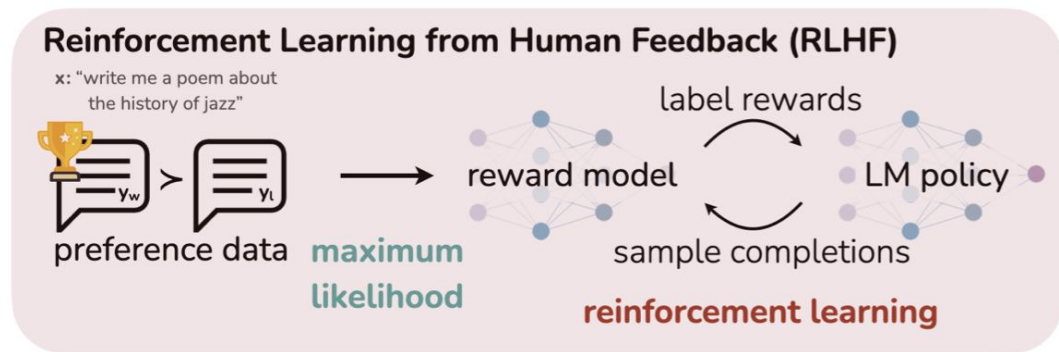
General Chat



Verifiable Tasks

Non-Verifiable Tasks

DPO (Direct Preference Optimization)



PEFT

How to train your Transformer at Home? (almost)



How much GPUs do you need?

Ful training (fp16):

Model parameters: 2 bytes/param

Optimizer states (AdamW): 8 bytes/param

Gradients: 2 bytes/param

Master copy: 4 bytes/param

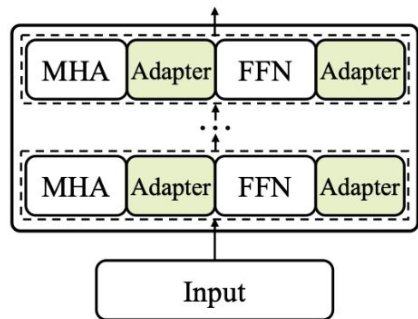
- **Full-training:** for 32B model: $\sim(2+8+2+4)*32B = 512 \text{ Gb}$ (7xA100)
- **Lora-training:** $\sim 60\text{Gb}$ (1xA100)

The rule of thumb

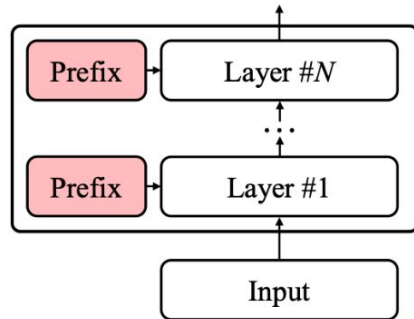
Train (fp16): 16GB of GPU memory per 1B parameters in the model

Inference: 2GB of GPU memory per 1B parameters in the model

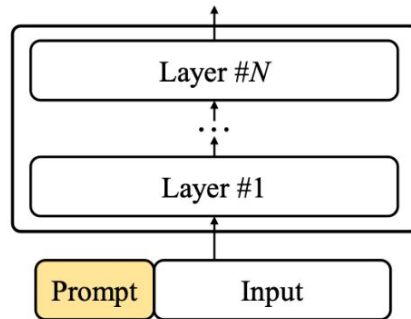
PEFT: Parameter Efficient Tuning



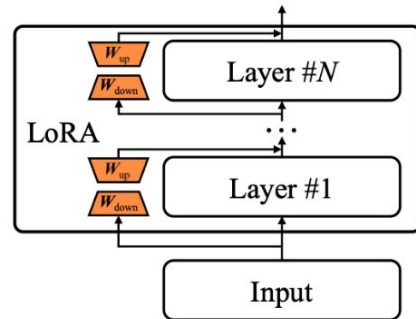
(a) Adapter Tuning



(b) Prefix Tuning



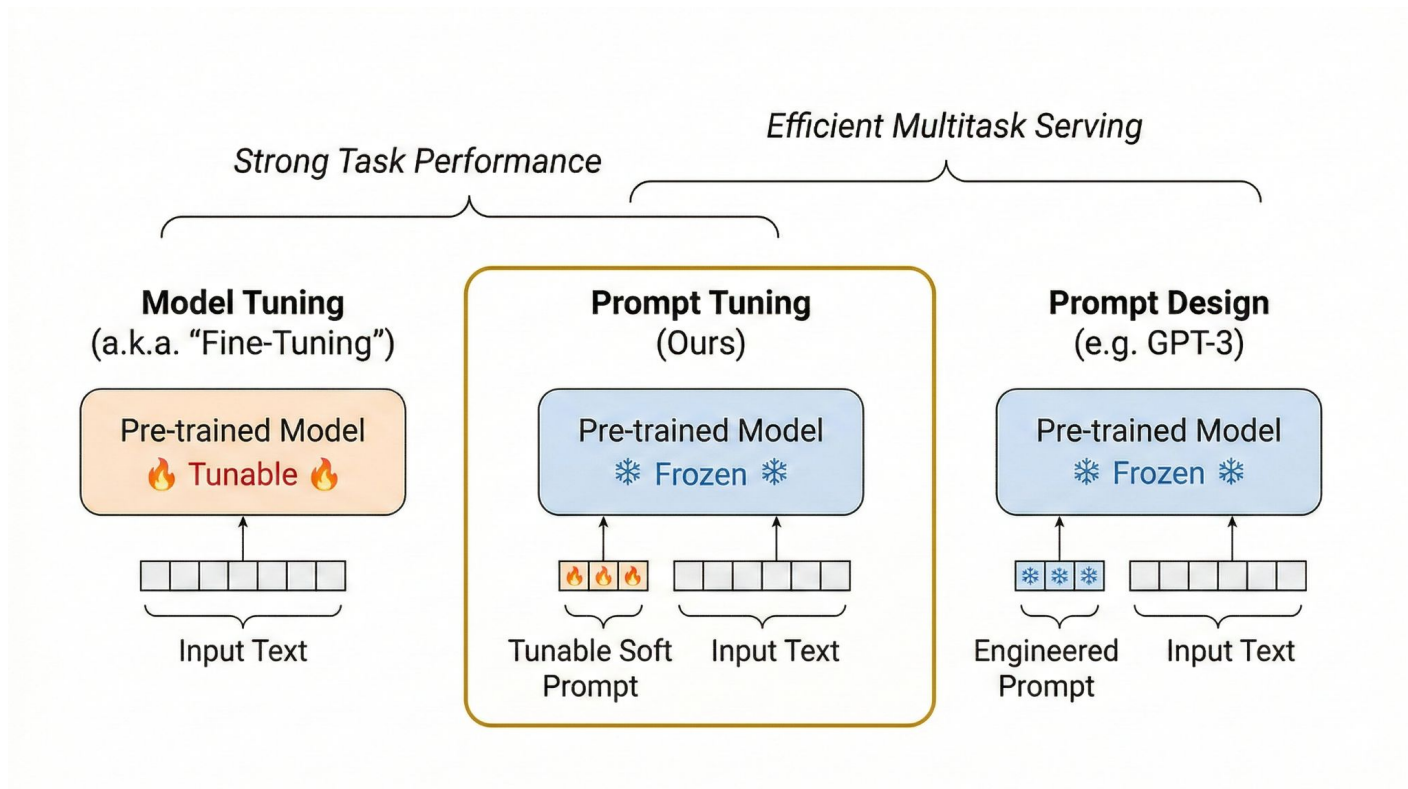
(c) Prompt Tuning



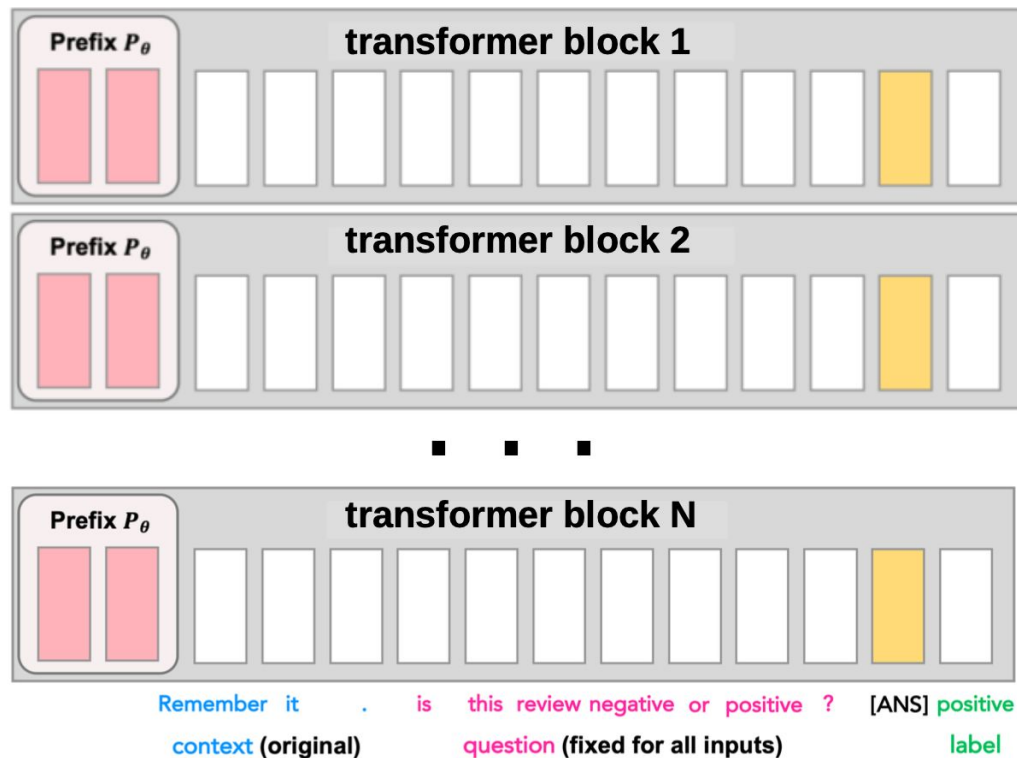
(d) Low-Rank Adaptation

Let's train specific model parts instead of Full Transformer!

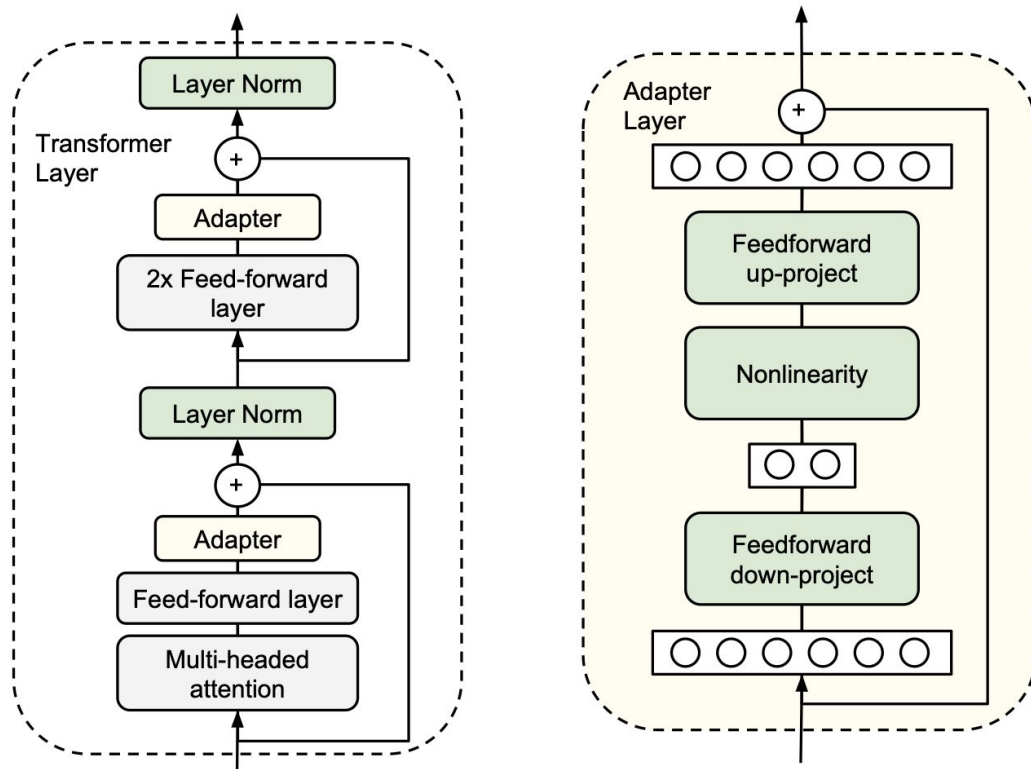
Prompt Tuning



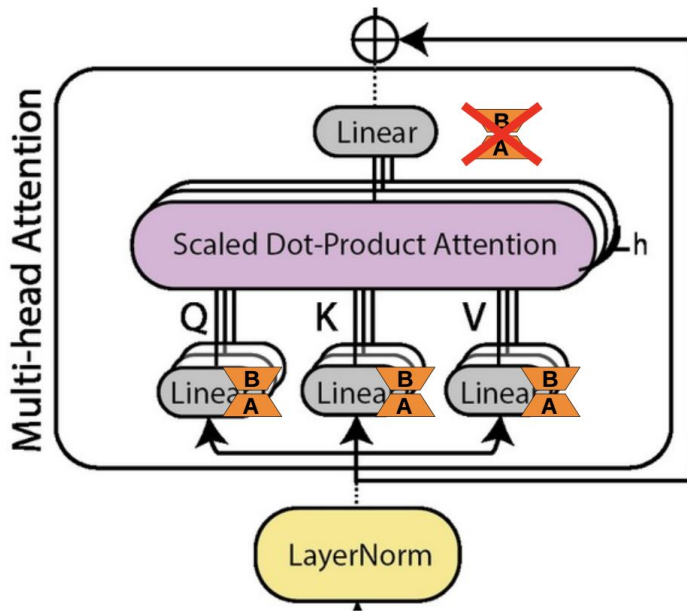
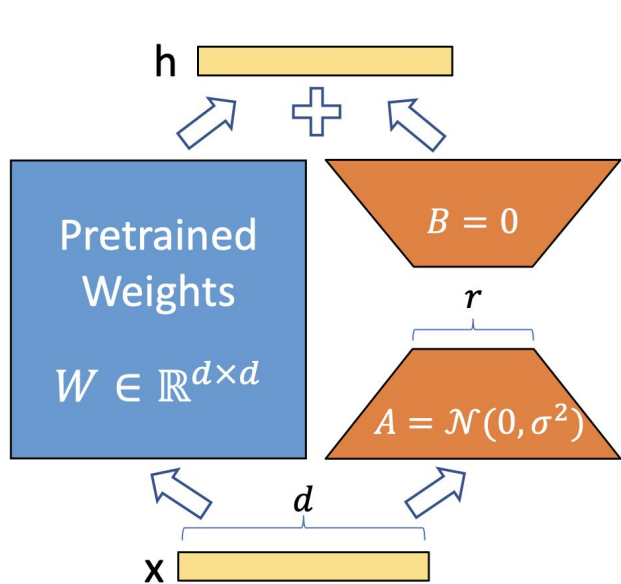
Prefix Tuning: deep prompt tuning



Adapters: train small sub-networks



LoRA: add adapters in parallel



Add LoRA to all Linear Layers, but not to Embeddings, MoE, Logits

LoRA: add adapters in parallel

VRAM requirements table

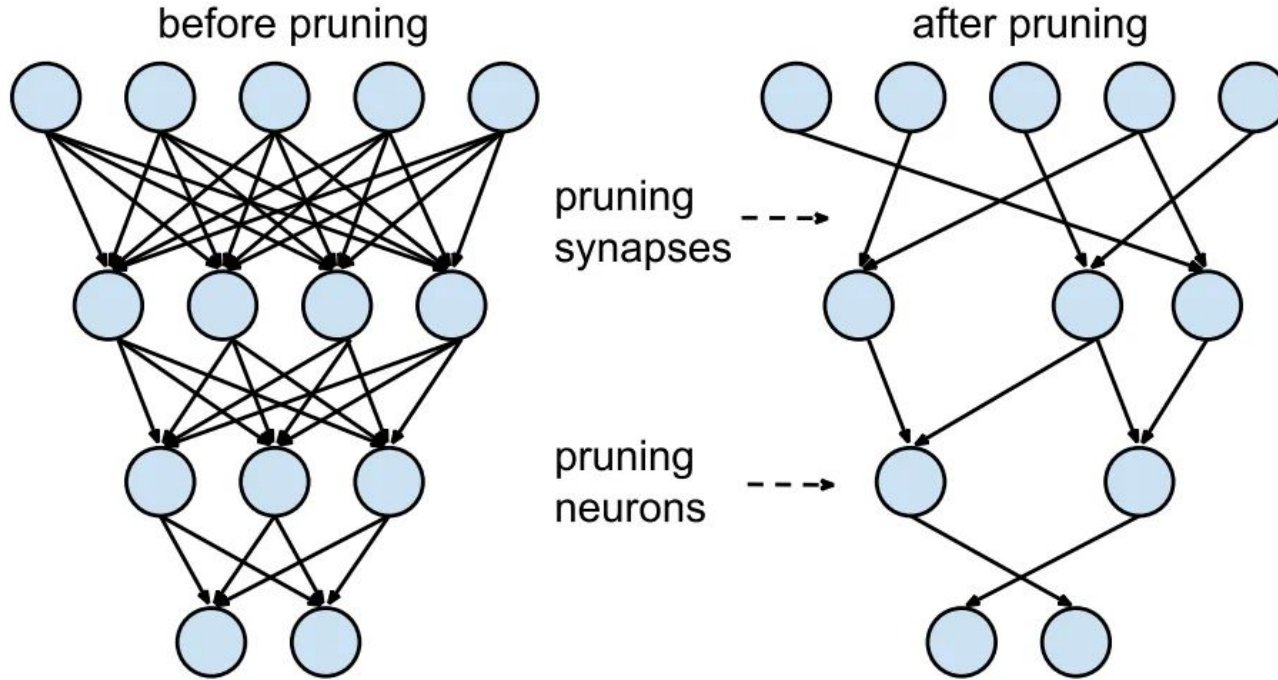
Here's a comparison of VRAM requirements for different model sizes and fine-tuning techniques:

Method	Precision	7B	13B	30B	70B	110B
Full	16	67GB	125GB	288GB	672GB	1056GB
LoRA	16	15GB	28GB	63GB	146GB	229GB
QLoRA	8	9GB	17GB	38GB	88GB	138GB
QLoRA	4	5GB	9GB	20GB	46GB	72GB

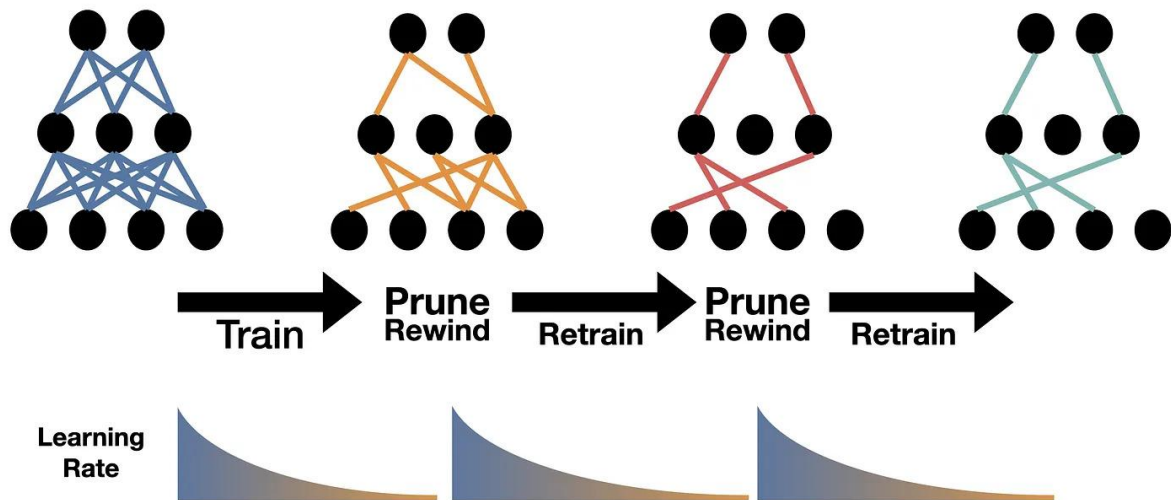
Note: These are approximate values and actual usage may vary based on implementation details and additional memory needs during training.

Pruning

Compression by pruning



Compression by pruning

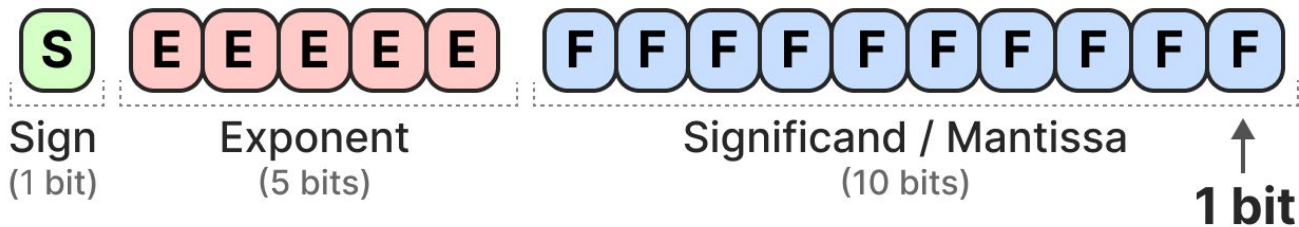


Drop ~5% smallest weights from each layer every 1000 steps (and keep training)

How computer
store model
weights?

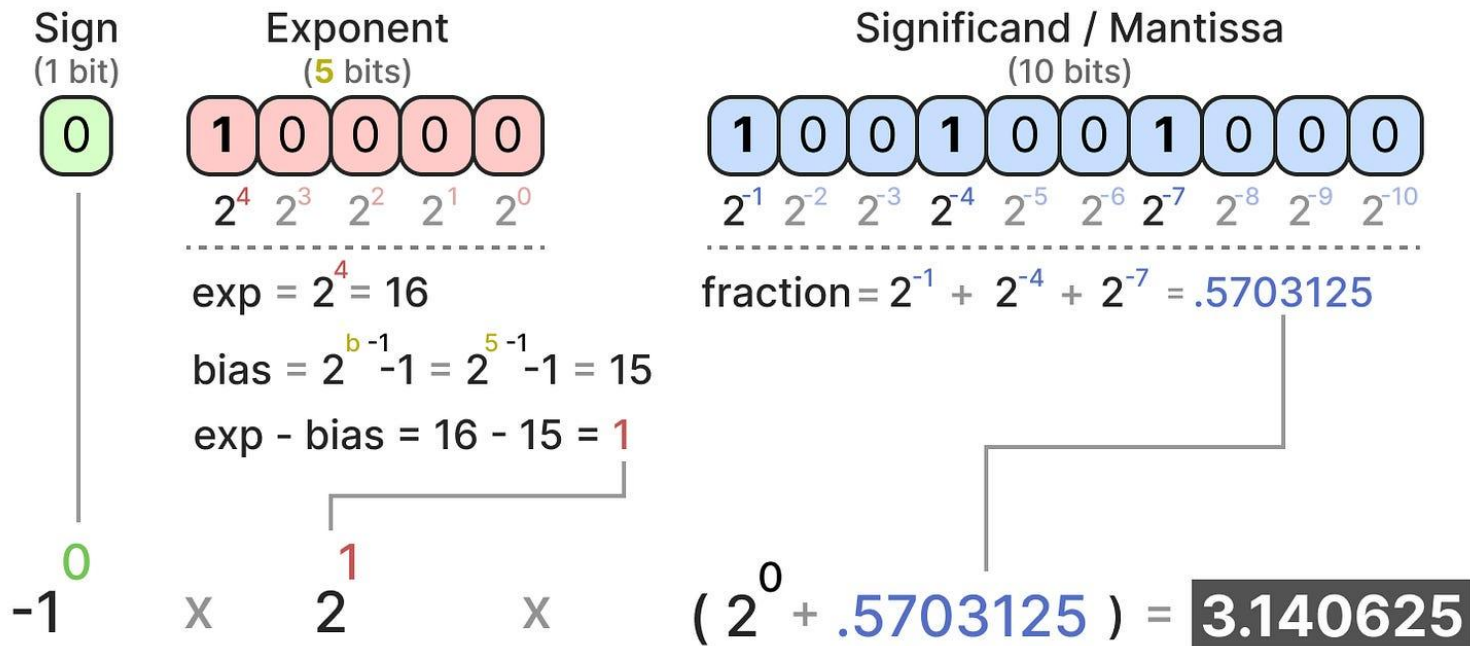
How computer store model weights?

Float 16-bit (FP16)



How computer store model weights?

Float 16-bit (FP16)



How computer store model weights?

Float 32-bit (FP32)

0 1 0 0 0 0 0 0 0 0 1 0 0 1 0 0 1 0 0 0 0 1 1 1 1 1 1 0 1 1 0 1 1

$$(-1)^0 \times 2^1 \times 1.5707964 = 3.1415927410125732$$

higher precision

Float 16-bit (FP16)

0 1 0 0 0 0 1 0 0 1 0 0 1 0 0 0

$$(-1)^0 \times 2^1 \times 1.5703125 = 3.140625$$

lower precision

original value
3.1415927

How computer store model weights?

Original	75505.0	1.8e-42
64-bits	75505.0	1.8e-42
32-bits	75505.0	1.80066e-42
16-bits	inf	0.0

How computer store model weights?

$$\text{memory} = \frac{\text{nr_bits}}{8} \times \text{nr_params}$$

$$\mathbf{64\text{-bits}} = \frac{64}{8} \times 70\text{B} \approx \mathbf{560\text{ GB}}$$

.....

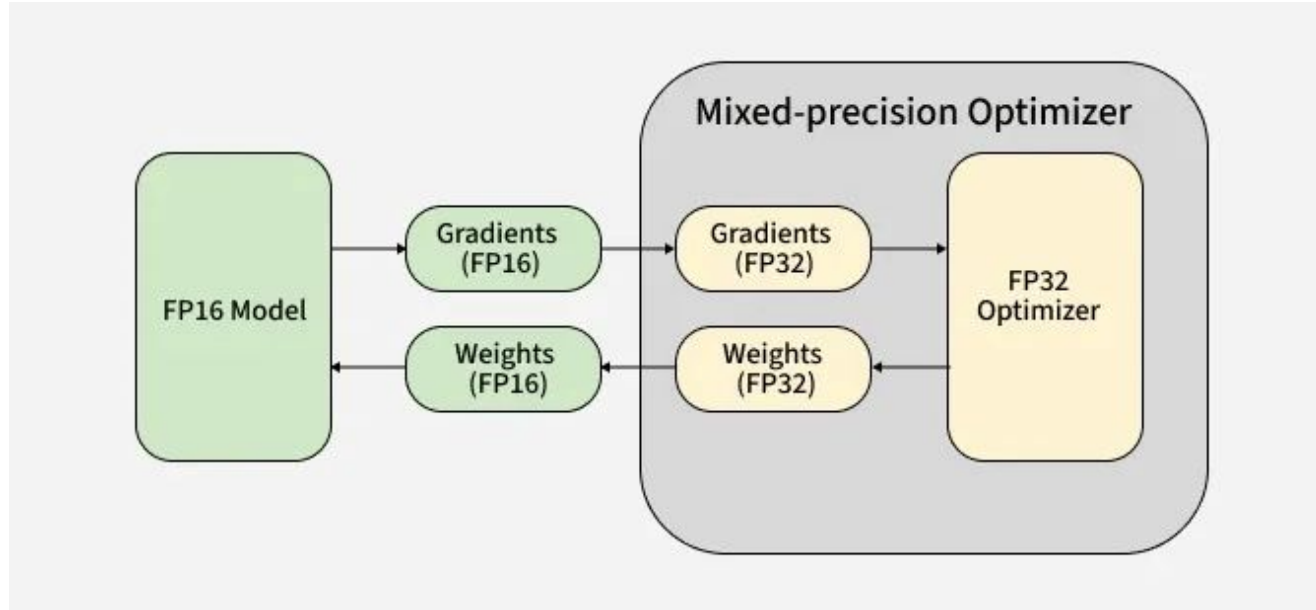
$$\mathbf{32\text{-bits}} = \frac{32}{8} \times 70\text{B} \approx \mathbf{280\text{ GB}}$$

.....

$$\mathbf{16\text{-bits}} = \frac{16}{8} \times 70\text{B} \approx \mathbf{140\text{ GB}}$$

Mixed Precision Training

Mixed Precision Training: what?

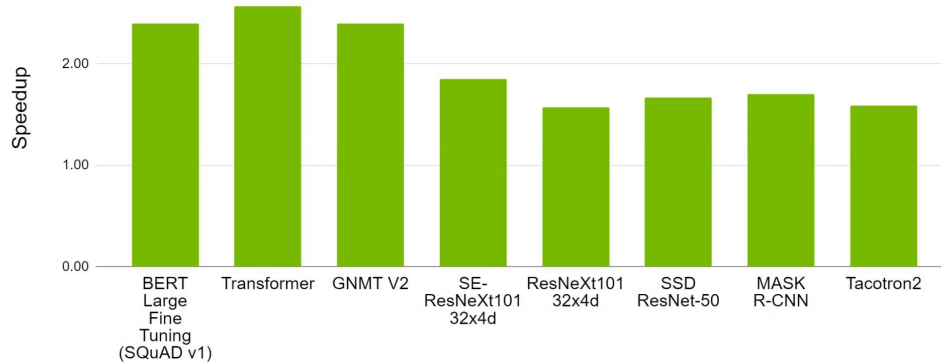


Mixed Precision Training: why?

Faster Training: Float16 operations execute faster on GPUs with specialized hardware (like NVIDIA Tensor Cores).

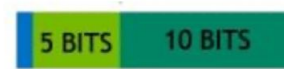
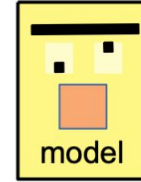
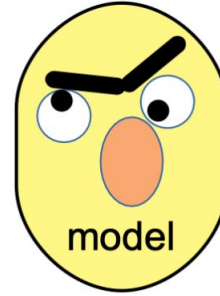
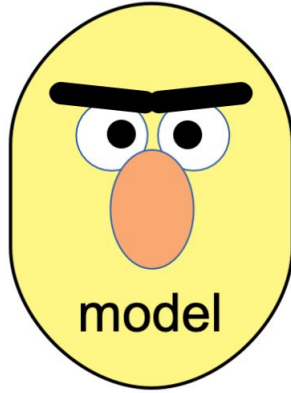
Lower Memory Usage: Reducing variable sizes from 32 bits to 16 bits nearly halves memory usage, enabling larger models or larger batch sizes.

Comparable Accuracy: When used with techniques like loss scaling, models trained with mixed precision perform similarly to full precision models



Quantization

Quantization



INT8

8 BITS

Quantization

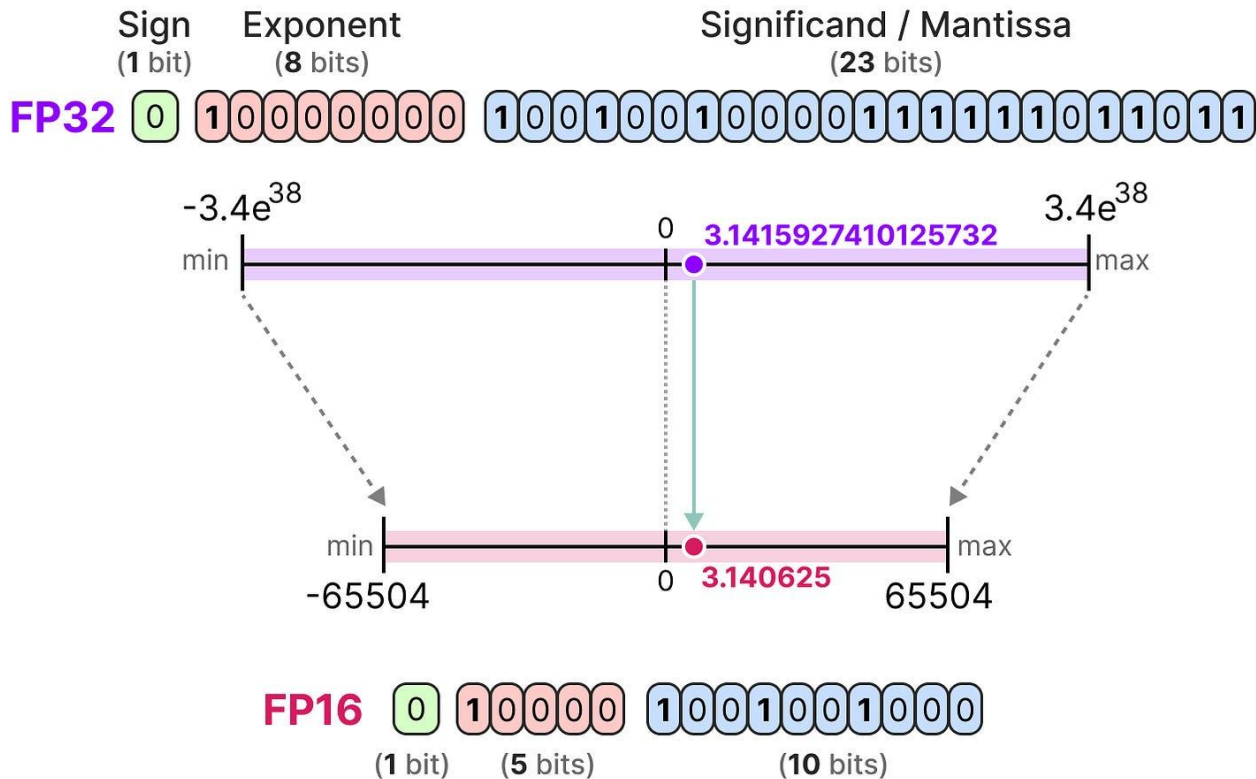
Original Image



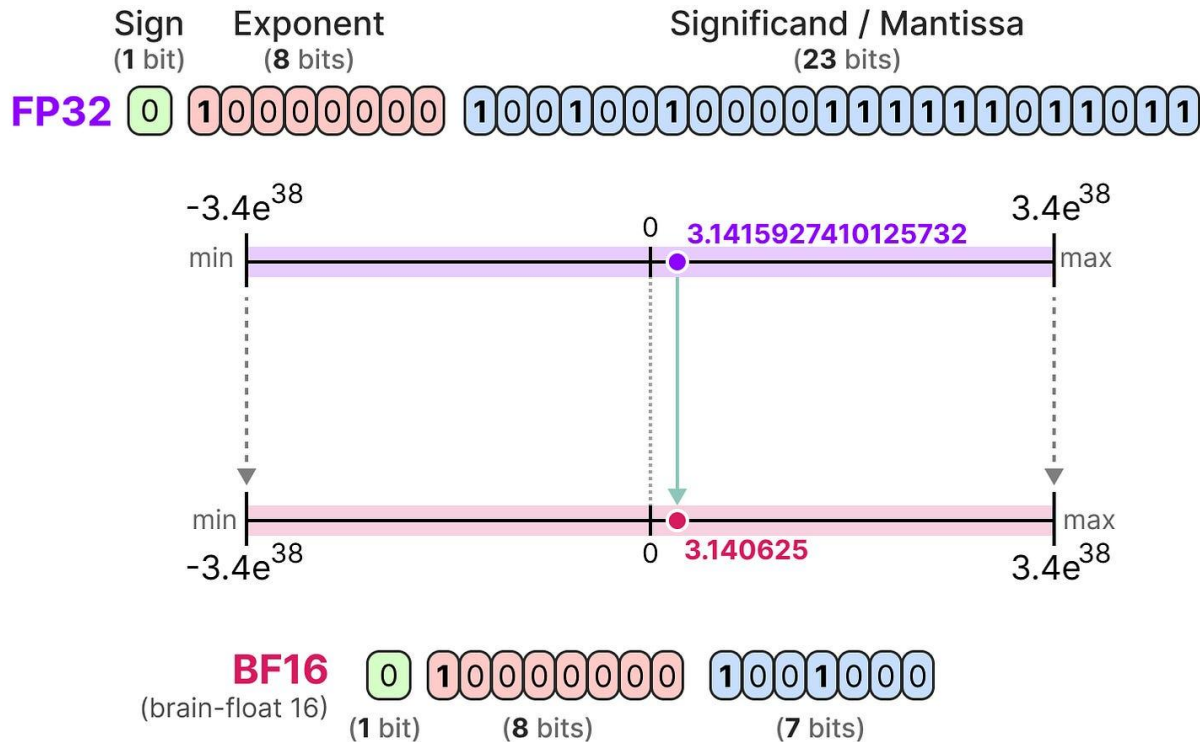
"Quantized" Image



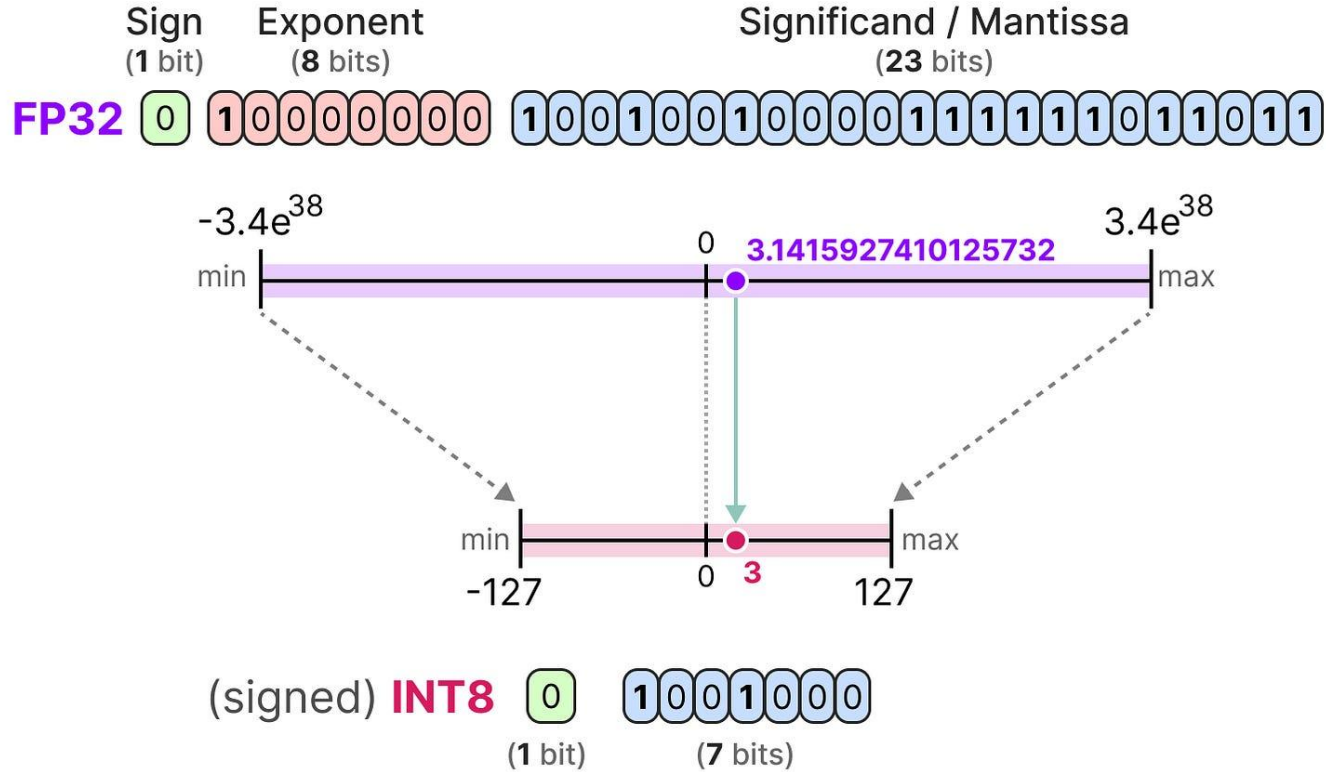
Quantization: fp16



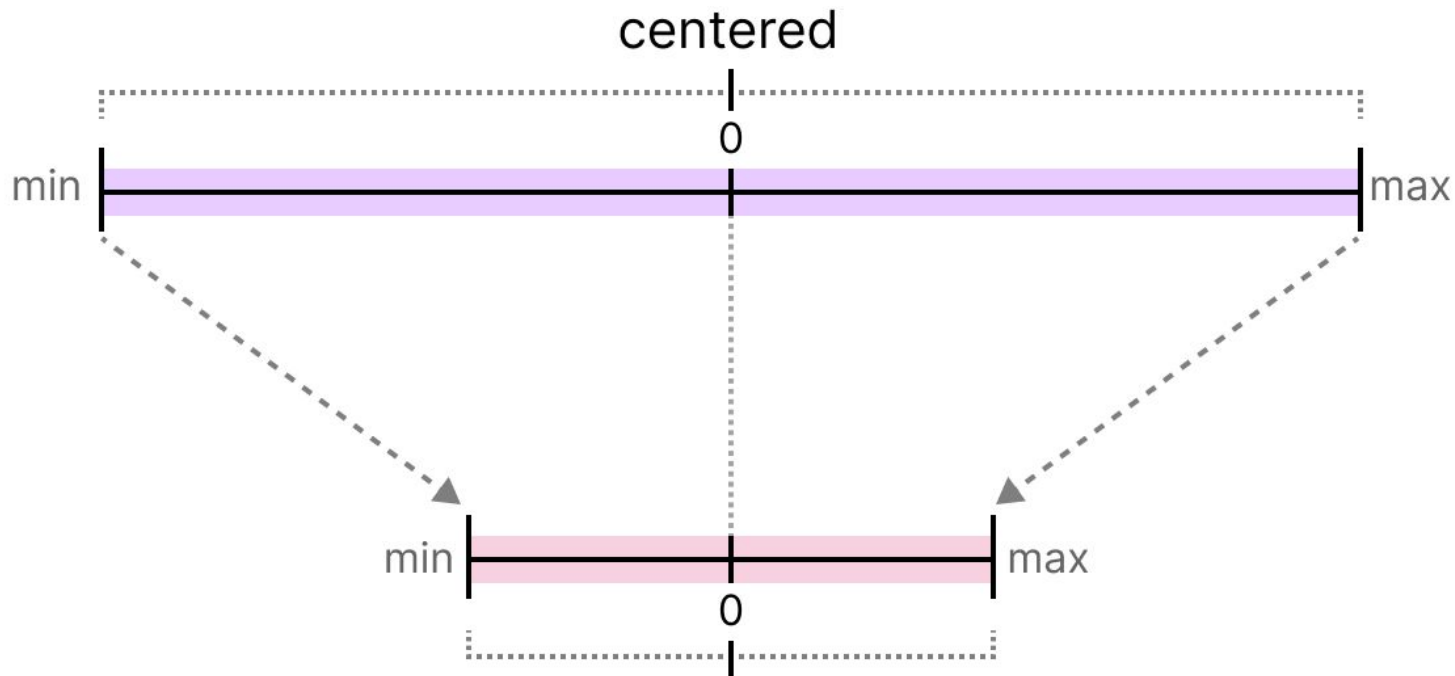
Quantization: bfloat16 (bf16)



Quantization: int8

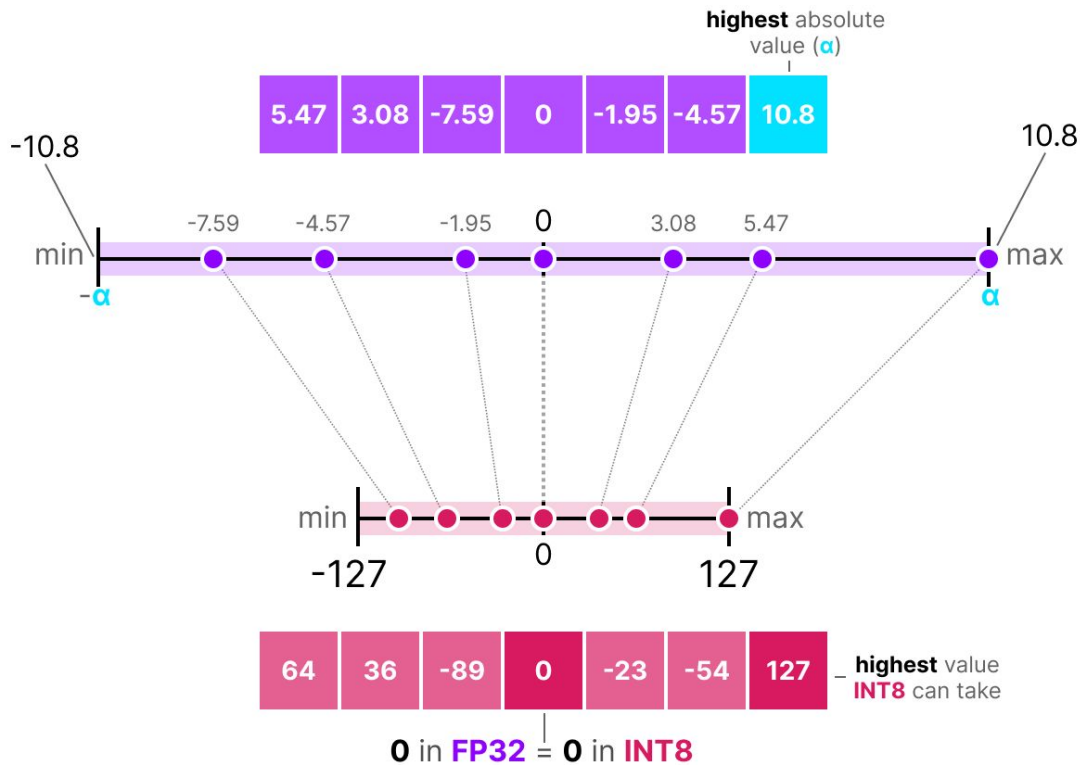


Quantization: centered int8



0 in **FP32** = **0** in **INT8**

Quantization: centered int8



Quantization: centered int8

$$s = \frac{2^{b-1} - 1}{\alpha} \quad (\text{scale factor})$$

$$x_{\text{quantized}} = \text{round}(s \cdot x) \quad (\text{quantization})$$

Filling in the values would then give us the following:

$$s = \frac{127}{10.8} = 11.76 \quad (\text{scale factor})$$

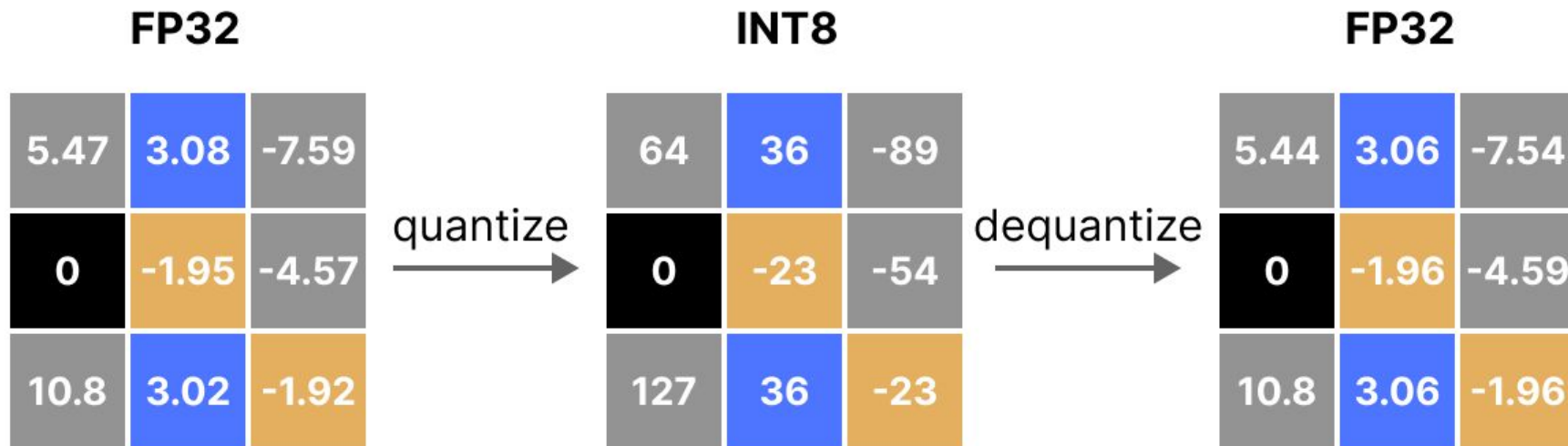
$$x_{\text{quantized}} = \text{round}(11.76 \cdot \text{[array of 5 purple squares]}) \quad (\text{quantization})$$

To retrieve the original FP32 values, we can use the previously calculated *scaling factor* (s) to *dequantize* the quantized values.

$$x_{\text{dequantized}} = \frac{\text{[array of 5 pink squares]}}{s} \quad (\text{dequantize})$$

- b is the number of bytes that we want to quantize to (8),
- α is the *_highest* absolute value,

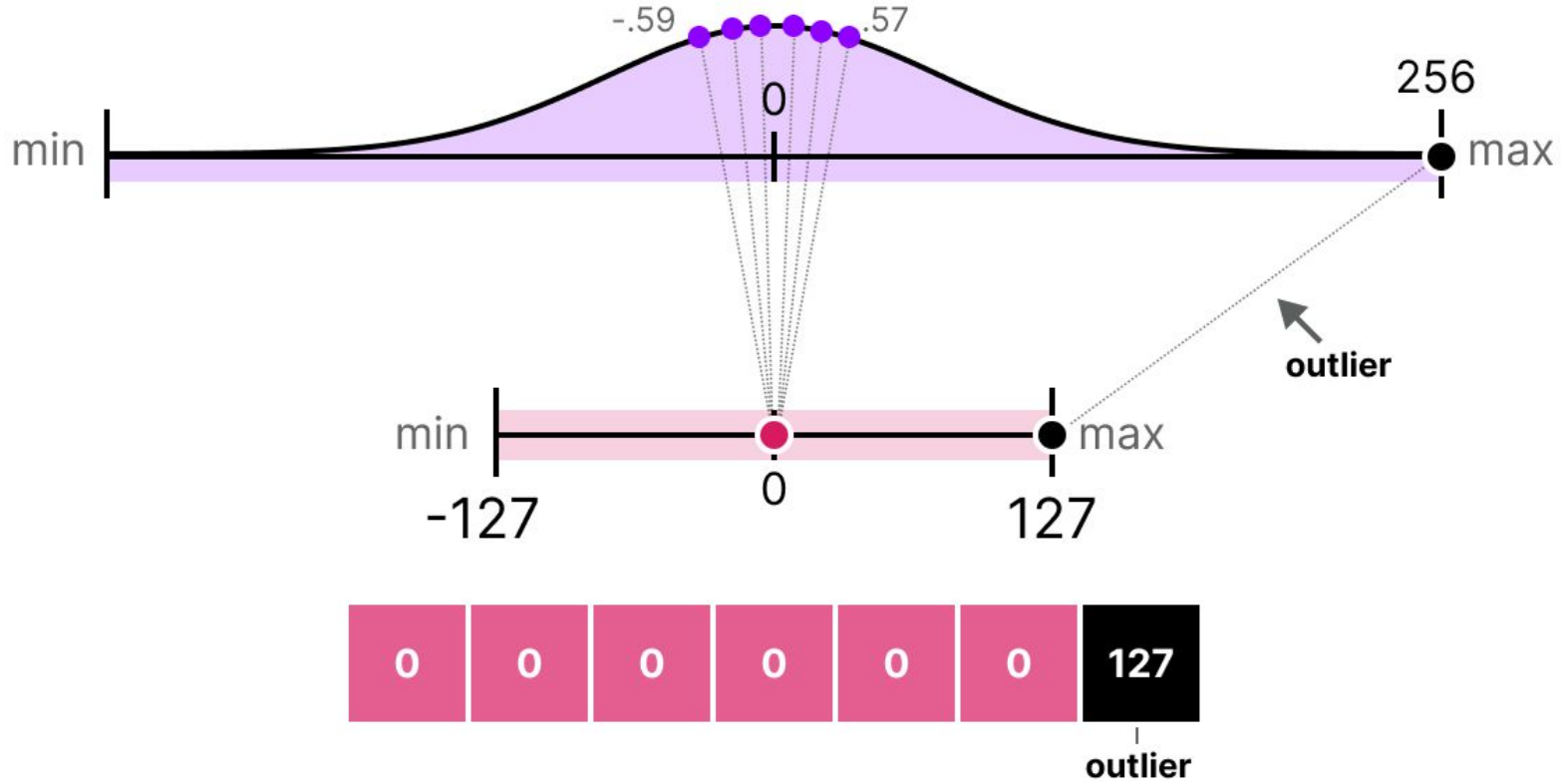
Quantization: centered int8



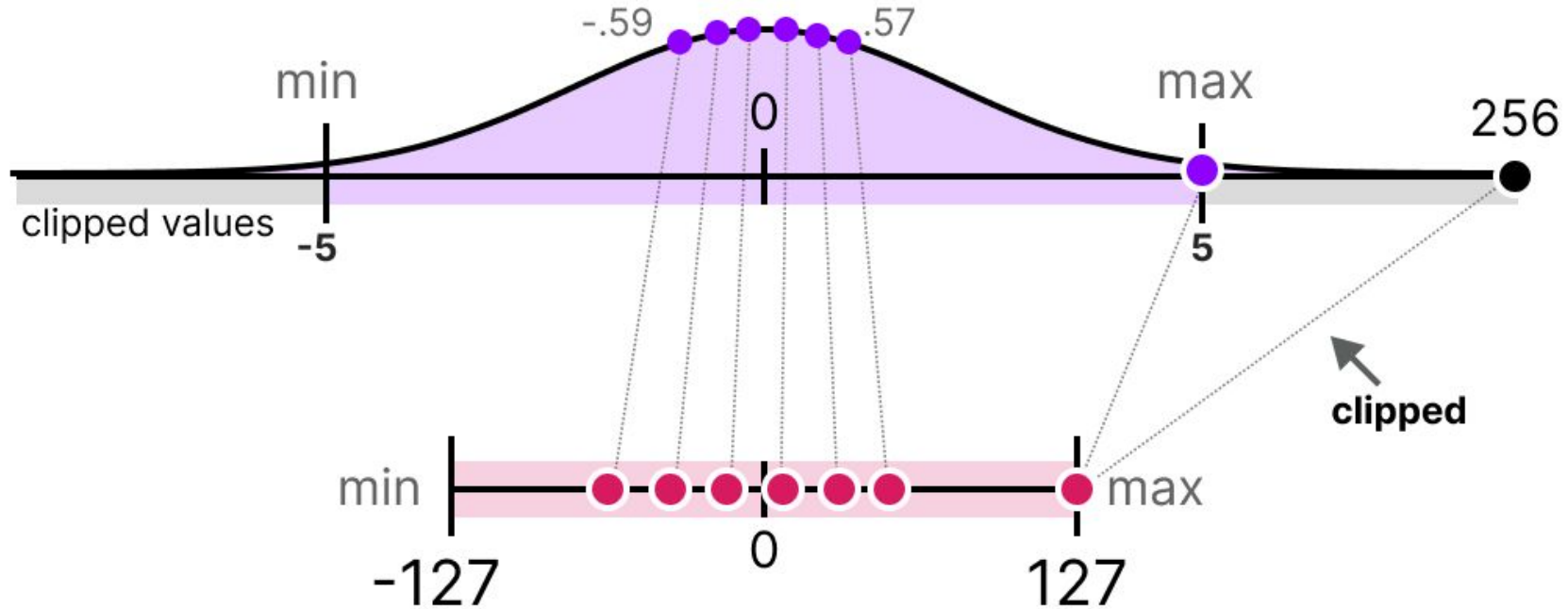
Quantization: quantization error

FP32 (original)				FP32 (dequantized)				Quantization error		
5.47	3.08	-7.59		5.44	3.06	-7.54		.03	.02	.05
0	-1.95	-4.57	-	0	-1.96	-4.59	=	0	-.01	-.02
10.8	3.02	-1.92		10.8	3.06	-1.96		0	-.04	-.04

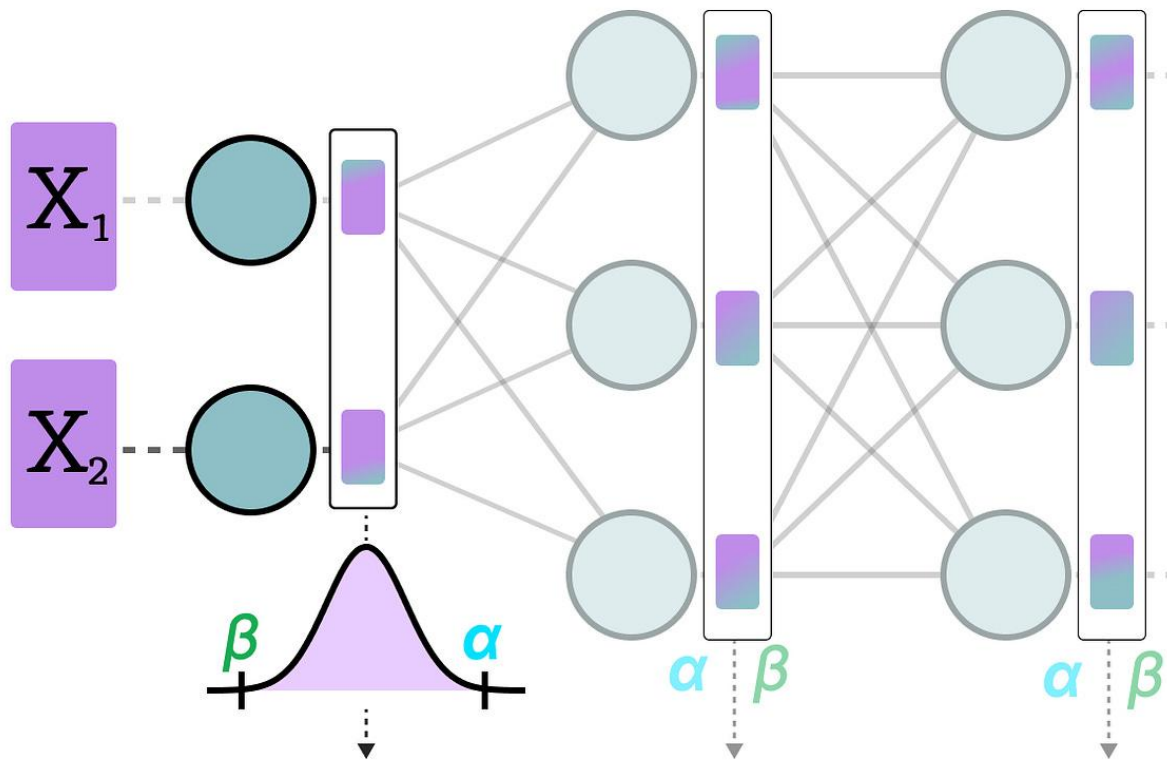
Quantization: outliers



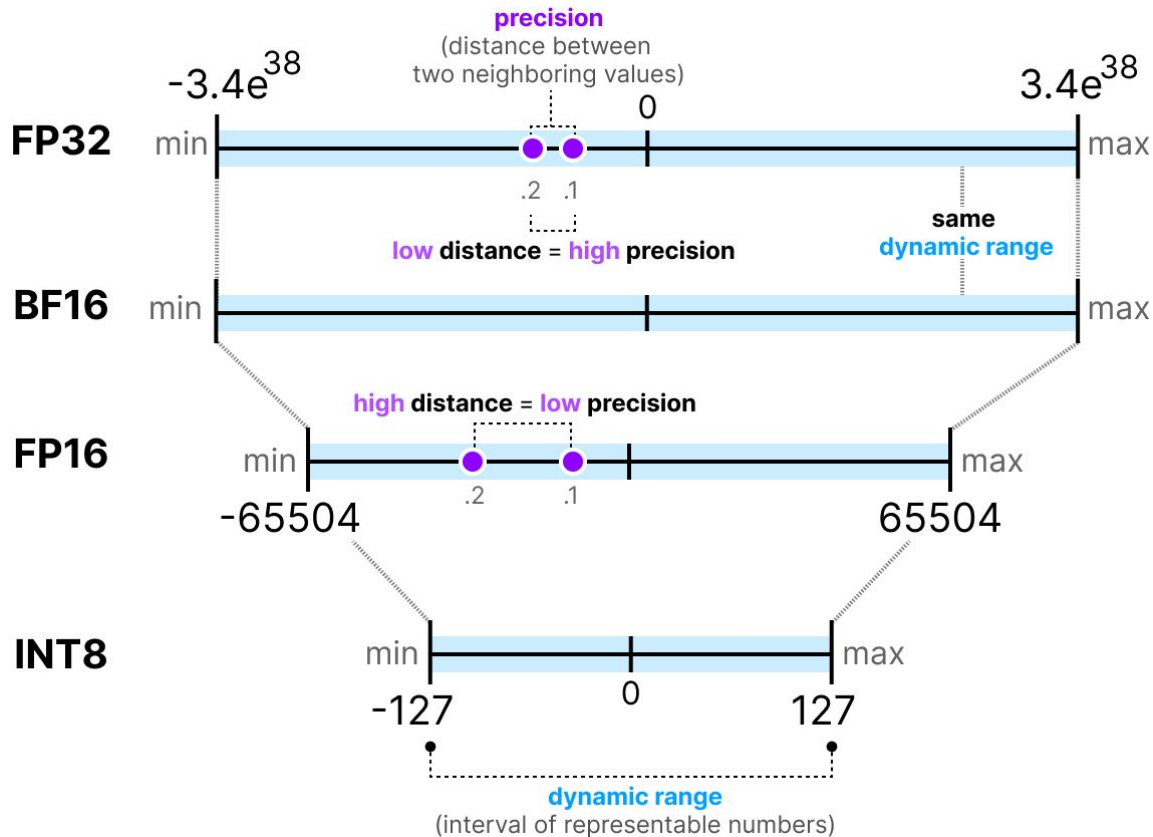
Quantization: range clipping



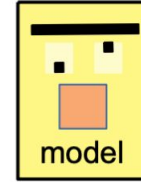
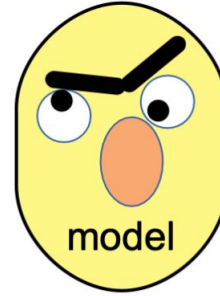
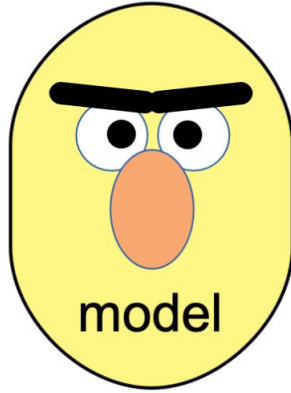
Quantization: model forward



Quantization



Quantization



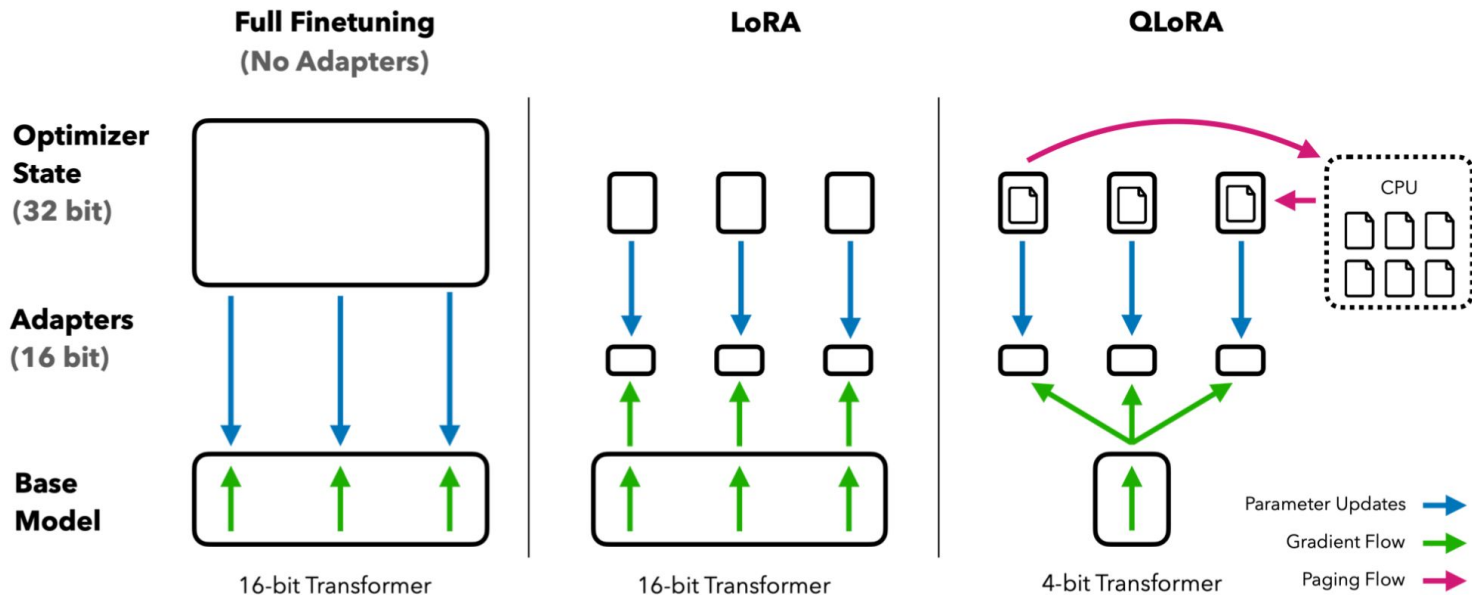
FP16



INT8

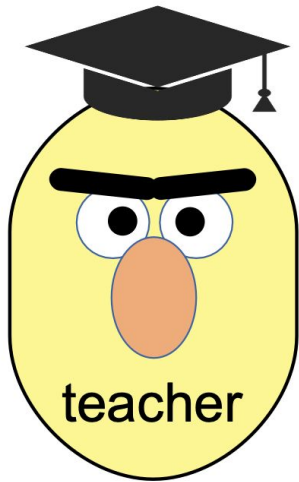


Quantized training: QLoRA



Distillation

Distillation: what if we need small domain-specific model?

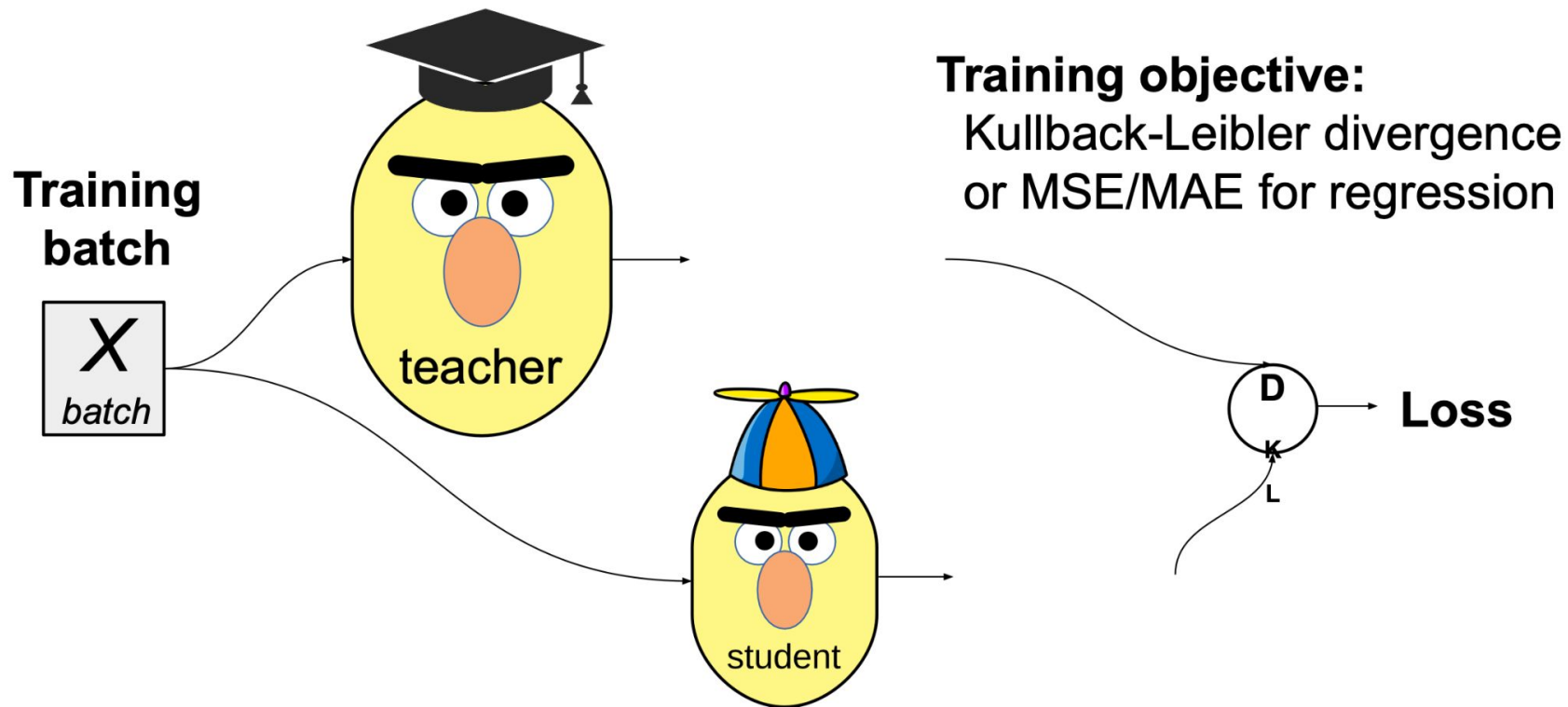


First, get the best performing model regardless of size



Then, train a more compact model to approximate it!

Distillation: what if we need small domain-specific model?



Distillation: student architecture choice

Naive: same but smaller, less layers / hidden units

- e.g. DistillBERT: <https://arxiv.org/pdf/1910.01108.pdf>
- Same as BERT-base, but with
- half as many layers
- (and ≈ 1.5 times faster)

Table 1: **DistilBERT retains 97% of BERT performance.** Comparison on the dev sets of the GLUE benchmark. ELMo results as reported by the authors. BERT and DistilBERT results are the medians of 5 runs with different seeds.

Model	Score	CoLA	MNLI	MRPC	QNLI	QQP	RTE	SST-2	STS-B	WNLI
ELMo	68.7	44.1	68.6	76.6	71.1	86.2	53.4	91.5	70.4	56.3
BERT-base	79.5	56.3	86.7	88.6	91.8	89.6	69.3	92.7	89.0	53.5
DistilBERT	77.0	51.3	82.2	87.5	89.2	88.5	59.9	91.3	86.9	56.3

Distillation: when it worth it?

- LLMs are trained up to compute budget
- Training the teacher (larger) model is VERY expensive
- Makes sense where:
 - Data is scarce, compute is not
 - A single teacher trains multiple students
 - Teacher achieves generalization and smaller model doesn't

Read more:

- Distillation Scaling Laws
- Puzzle: Distillation-Based NAS for Inference-Optimized LLMs